CNP

Programma del corso

Filosofia didattica

Githubification of InfoSec

L'intero ecosistema della sicurezza offensiva e difensiva vive su GitHub: framework, strumenti, TTPs, playbook, dataset di intelligence — tutto pubblicamente accessibile. Il vantaggio è la democratizzazione; lo svantaggio è che l'attaccante ha esattamente gli stessi strumenti del difensore, aggiornati in tempo reale.

La conseguenza didattica è diretta: gli studenti partono da un repository GitHub, leggono la documentazione, risolvono i problemi di installazione e configurazione da soli. L'attrito è parte dell'apprendimento.

L'automazione non è vietata — è incoraggiata. Se un team decide di automatizzare parti del proprio flusso operativo, incluso lo sviluppo agentico, è libero di farlo. L'unico vincolo è l'autonomia: ogni soluzione deve essere costruita dal team, compresa e difendibile.

Dimostrazione di padronanza Al termine di ogni modulo tutti i team dimostrano live di aver acquisito padronanza dello strumento introdotto. Non è una presentazione — è una verifica operativa: l'istruttore o i team avversari possono chiedere di eseguire qualsiasi operazione rientrante nel perimetro del modulo.

Economia operativa e scambio di capacità

L'ambiente di gioco non è solo tecnico ma anche strategico: le capacità operative (configurazioni, script, pipeline di detection, accessi) possono essere scambiate tra team.

Ogni team dispone di un budget iniziale di **Crediti Operativi (CO)**.

I crediti possono essere:

- **Guadagnati** vendendo capacità operative ad altri team;
- **Spesi** acquistando soluzioni, configurazioni o informazioni;
- **Persi** in caso di dipendenza eccessiva da soluzioni esterne.

Gli scambi possono riguardare:

- configurazioni funzionanti (es. Caldera, Sysmon);
- snippet API validati;
- regole di detection;
- pipeline di parsing o analisi log.

Ogni scambio comporta rischio: le soluzioni acquistate possono essere incomplete, non funzionare nel contesto specifico o introdurre pattern riconoscibili. Ogni capability venduta può essere riutilizzata contro il venditore.

L'istruttore può agire come **broker neutrale**, facilitando scambi anonimi e garantendo coerenza operativa.

L'obiettivo non è la cooperazione, ma la gestione strategica delle risorse in un ambiente competitivo.

Struttura di gioco — Tre team, tutti contro tutti

Il corso è strutturato come un esercizio continuativo a tre team con ruoli simultanei. Ogni team è contemporaneamente:

- **Red Team** verso gli altri due: esegue TTPs su Caldera seguendo fedelmente il profilo di un attore reale assegnato, che gli altri non conoscono;
- **Blue Team** verso gli altri due: vede solo ciò che emerge dal proprio stack di rilevamento, senza sapere quanti attori sta fronteggiando né chi sono.

Ogni team deve quindi attaccare su due fronti, difendere su due fronti, e — progressivamente — costruire un ambiente di deception senza sapere da quale direzione arriveranno le esche toccate. L'attribuzione degli attori avversari in tempo reale è parte integrante del gioco.

Quadro normativo

Breve ancoraggio legale prima di operare: l'**art. 615-ter c.p.** definisce il confine tra difesa attiva lecita e accesso abusivo a sistema informatico. Gli studenti cercano autonomamente le fonti disponibili — la letteratura online è limitata — e producono collettivamente una mappa sintetica dei confini operativi in Italia. L'output è un documento di una pagina, non un'analisi esaustiva.

Modulo 1 — Cyber Threat Intelligence & ATT&CK

Concetti da scoprire

- La struttura di MITRE ATT&CK: Tattiche (obiettivo) e Tecniche/Sub-tecniche (esecuzione);
- Come un report CTI reale si traduce in TTPs mappabili.

Ruoli interni

Prima di avviare qualsiasi attività, ogni team definisce e dichiara i propri ruoli operativi interni: chi opera come red, chi come blue, chi come analyst. I ruoli possono essere condivisi o turnati, ma devono essere espliciti e documentati nel log.

Lab (es. APT29, Lazarus Group, Sandworm). Lo studia attraverso le seguenti fonti pubbliche:

- **MITRE ATT&CK Groups** — profilo dell'attore con TTPs associate e riferimenti alle fonti primarie;
- **MITRE CTI** — repository GitHub ufficiale di MITRE con i dati ATT&CK in formato STIX. Permette di interrogare programmaticamente l'intero dataset: estrarre TTPs, relazioni tra tecniche, software e gruppi in modo strutturato anziché navigare manualmente l'interfaccia web;
- **Report vendor** — Mandiant, CISA Advisory, Microsoft MSTIC, Secureworks;
- **MITRE Engenuity / Center for Threat-Informed Defense** — piani di emulazione avversariale dettagliati, disponibili su GitHub, con procedure step-by-step per ogni TTP.

Dall'insieme di queste fonti il team ricostruisce la kill chain completa in due formati complementari:

- **ATT&CK Navigator** — mappa di presenza delle TTPs, utile per visualizzare la copertura complessiva dell'attore;
- **Attack Flow** — linguaggio del Center for Threat-Informed Defense per rappresentare la *sequenza causale* dei comportamenti avversariali. Non solo quali TTPs usa l'attore, ma in quale ordine e con quale dipendenza logica tra un'azione e la successiva. È il documento che guida la fedeltà operativa in Caldera.

Il team definisce un **vincolo di coerenza operativa**: nei moduli successivi dovrà operare in modo credibile rispetto al proprio attore, seguendo la logica e le dipendenze dell'Attack Flow.

L'uso di TTPs non documentate è consentito ma soggetto a penalità, in base al grado di deviazione dal profilo dell'attore.

Operation log

Ogni team red è tenuto a mantenere un **operation log continuo e timestampato** per tutta la durata del corso. Il log deve registrare in tempo reale ogni azione eseguita: TTP, strumento usato, timestamp, esito, motivazione operativa.

Il log è la **fonte di verità esclusiva** per il calcolo del punteggio e per il reveal del Modulo 6. Non è ammesso ricostruirlo a posteriori.

Requisiti tecnici:

- il log deve risiedere in un repository git con commit progressivi durante le operazioni — il timestamp dei commit è parte integrante della verifica;
- il repository deve essere privato ma accessibile in sola lettura all'istruttore fin dal Modulo 2;
- l'identità del team (e in particolare l'attore assegnato) non deve essere desumibile dal nome del repository o dall'account prima del reveal.

Il punto di riferimento suggerito è **Codeberg** — hosting git open source, senza tracciamento, senza carta di credito, account pseudonimo sufficiente. Ogni team è libero di adottare una soluzione alternativa purché garantisca gli stessi requisiti: immutabilità dei timestamp, accesso controllato, nessuna fuga di identità prima del reveal.

Ogni team gestisce e archivia il proprio lavoro come preferisce. La mappa deve rimanere privata fino al Modulo 6.

Output: Mappa ATT&CK e Attack Flow per l'attore assegnato, privati fino al reveal.

Coerenza di attore e penalità

Durante le operazioni, ogni azione viene valutata in base alla coerenza con il profilo assegnato:

- **TTP coerente (documentata)** → punteggio pieno
- **TTP plausibile (non documentata ma compatibile)** → nessun bonus
- **TTP fuori profilo** → penalità
- **Sequenza incoerente con Attack Flow** → penalità maggiore

Ogni team dispone inoltre di un numero limitato di **TTP jolly**, utilizzabili per introdurre variazioni controllate senza compromettere la coerenza complessiva.

L'obiettivo non è la rigidità, ma la credibilità operativa.

Modulo 2 — Adversary Emulation con Caldera

Concetti da scoprire

- Struttura di Caldera: profilo avversario, agente, operazione.

Lab

Fase 1 — Infrastruttura target Ogni team installa e configura autonomamente almeno una **VM Windows 10** da usare come sistema target.

La configurazione di rete non è una scelta banale: **Bridge** espone la VM sulla rete reale dell'ambiente ed è necessario per la raggiungibilità tra i tre team; **NAT** la isola dietro l'host, semplifica la gestione ma può impedire l'accesso incrociato a Caldera. La scelta impatta direttamente l'intera architettura del gioco e va presa consapevolmente, non lasciata al default del hypervisor.

Fase 2 — Training Caldera Prima di qualsiasi operazione reale, ogni team completa il **modulo di training integrato di Caldera** — il percorso di auto-apprendimento guidato disponibile direttamente nella piattaforma. È un prerequisito non negoziabile: nessun team avvia operazioni senza averlo completato e dimostrato.

Fase 3 — Setup operativo Ogni team clona e installa Caldera da GitHub, fa il deploy almeno dell'agente Sandcat sulla VM Windows 10 target, e configura un profilo operativo che riproduce le TTPs del proprio attore assegnato.

Fase 4 — Stack di rilevamento Ogni team sceglie e configura il proprio stack di rilevamento. Il punto di riferimento consigliato è la combinazione **Sysmon + Sigma + Chainsaw/Hayabusa**: leggera, interamente su GitHub, vicina a come lavora un analista reale su un endpoint e adatta all'automazione agentica. Ogni team è libero di adottare lo stack che preferisce. ELK o commerciale.

Fase 5 — Accesso incrociato Ogni team espone la propria istanza Caldera agli altri due team avversari, che la useranno per attaccare il proprio ambiente (quello messo a disposizione). Il problema è che in Caldera gli operatori red che condividono la stessa istanza possono vedere le operazioni altrui. Ogni team sceglie autonomamente come gestirlo tra tre opzioni:

- **Istanze separate** — ogni team gira un'istanza Caldera dedicata per ogni ambiente che attacca. Nessuna visibilità incrociata, ma triplica il carico infrastrutturale;
- **Accesso via API** — ogni team espone la propria istanza solo tramite API key, senza accesso alla UI. Gli avversari inviano operazioni via chiamate REST. Nessuna visibilità sulle operazioni altrui, più aderente a un contesto reale. Caldera espone una REST API

completamente documentata via Swagger UI sull'istanza in esecuzione (`/api/docs`) — progettata per essere interrogata e automatizzata agenticamente;
- **Scope separato per operatore** — configurazione delle operazioni per gruppi distinti all'interno della stessa istanza. Meno isolamento garantito, dipende dalla versione in uso.

Nota operativa — Variabilità degli ambienti

Ogni ambiente operativo può differire significativamente per:

- configurazione di rete;
- versione degli strumenti;
- compatibilità tra componenti;
- comportamento degli agenti.

I team possono scegliere se:

- costruire l'ambiente da zero;
- partire da configurazioni parziali;
- adattare componenti già esistenti.

La scelta influisce sul tempo operativo, sulla comprensione del sistema e sull'esposizione a errori.

Non esiste una configurazione "standard": ogni soluzione deve essere compresa e difendibile.

Checkpoint tecnico obbligatorio: tutti e tre i team devono avere Caldera operativo, Windows 10 target raggiungibile e almeno un'operazione completata con successo. Lo scontro a tre fronti inizia solo da questo punto.

Nota — Esempio di chiamata API REST a Caldera

Il codice seguente è un punto di partenza per chi sceglie l'accesso via API. Non è detto che funzioni — anzi. Versioni diverse di Caldera, configurazioni di rete, permessi e struttura dei payload cambiano. Solo il confronto con un agente, con il prompt giusto, può dare la risposta corretta per il proprio ambiente specifico.

```
import requests
import json
#
==============================================================================
```

```
# SCRIPT DI ESEMPIO: LANCIO OPERAZIONE CALDERA VIA API REST
# Modulo 2: Adversary Emulation con Caldera
#
==============================================================================
# 1. CONFIGURAZIONE DEL TARGET E DELL'AUTENTICAZIONE
# Sostituite questi valori con l'IP della vostra istanza Caldera e la vostra
API KEY.
# L'API Key in Caldera funge da token di autorizzazione primario per le
chiamate REST.
CALDERA_IP = "192.168.1.100"
CALDERA_PORT = "8888"
API_KEY = "INSERISCI_QUI_LA_TUA_API_KEY_ROSSA" # Trovi la chiave nella UI web
o nel file conf.yml
# Costruiamo l'URL base dell'API (versione 2)
BASE_URL = f"http://{CALDERA_IP}:{CALDERA_PORT}/api/v2"
# 2. DEFINIZIONE DEGLI HEADERS
# Questi sono i "metadati" della nostra richiesta HTTP.
# Caldera richiede esplicitamente l'header "KEY" per l'autenticazione.
headers = {
    "KEY": API_KEY,                                # Il nostro pass per entrare
    "Content-Type": "application/json",            # Diciamo a Caldera che gli
invieremo dati in formato JSON
    "Accept": "application/json"                   # Ci aspettiamo una risposta
in formato JSON
}
# 3. COSTRUZIONE DEL PAYLOAD (IL CORPO DELLA RICHIESTA)
# Qui definiamo i parametri operativi. Per lanciare un'operazione, dobbiamo
dire a Caldera
# quale attore stiamo emulando e come vogliamo che venga pianificato
l'attacco.
# I valori ID (adversary_id, planner_id) vanno recuperati prima via API o
interrogando la UI.
payload = {
    "name": "Operazione_APT29_Infiltration", # Date un nome riconoscibile
all'operazione
    # ADVERSARY: Definisce *cosa* farete. È il profilo che contiene le TTPs
    # che avete mappato nel Modulo 1 per il vostro attore specifico.
    "adversary": {
        "adversary_id": "456789-abcde-12345" # Inserite qui l'ID esatto del
vostro profilo avversario
    },
    # PLANNER: Definisce *come* verranno eseguite le TTPs.
    # Es. "atomic" (esegue tutto in ordine), "batch", ecc.
    "planner": {
        "planner_id": "123456-fghij-67890" # Inserite l'ID del planner scelto
    },
```

```
    # STATE: "running" fa partire immediatamente l'operazione sugli agenti
(es. Sandcat) attivi.
    "state": "running",
    # JITTER: Importante per l'evasion. Aggiunge un ritardo casuale (es. tra 2
e 5 secondi)
    # tra l'esecuzione di una TTP e l'altra per sembrare meno "robotici" agli
occhi del Blue Team avversario.
    "jitter": "2/5",
    # VISIBILITY: 50 è il valore predefinito. Simula quanto rumore fa
l'agente.
    "visibility": 50
}
# 4. ESECUZIONE DELLA CHIAMATA API
# Usiamo il metodo POST perché stiamo *creando* una nuova risorsa
(l'operazione) sul server.
endpoint = f"{BASE_URL}/operations"
print(f"[*] Tentativo di lancio operazione su {endpoint}...")
try:
    response = requests.post(url=endpoint, headers=headers,
data=json.dumps(payload), timeout=10)
    # 5. GESTIONE DELLA RISPOSTA (TROUBLESHOOTING)
    if response.status_code in [200, 201]:
        print("[+] Successo! Operazione lanciata correttamente.")
        dati_operazione = response.json()
        print(f"[*] ID Operazione assegnato: {dati_operazione.get('id')}")
    elif response.status_code in [401, 403]:
        print("[-] Errore di Autenticazione: Controllate la vostra API_KEY.")
    else:
        print(f"[-] Errore {response.status_code}: Impossibile lanciare
l'operazione.")
        print(f"Dettagli: {response.text}")
except requests.exceptions.ConnectionError:
    print("[-] Errore di Connessione: Impossibile raggiungere l'IP di Caldera.
Controllate il routing o la VPN.")
except Exception as e:
    print(f"[-] Si è verificato un errore inaspettato: {e}")
```

Output: Ambiente target operativo; profilo Caldera configurato; stack di rilevamento attivo.

Modulo 3 — Contromisure con D3FEND

Concetti da scoprire

- Struttura di MITRE D3FEND: primitive Harden, Detect, Isolate, Deceive, Evict;
- Relazione bidirezionale tra tecniche ATT&CK e contromisure D3FEND;
- Prioritizzazione delle contromisure in base ai gap di visibilità osservati.

Lab

Ogni team analizza gli eventi accumulati dal proprio stack di rilevamento e identifica i **buchi di visibilità**: TTPs passate inosservate. Non sa ancora da quale attore provengano.

Naviga la matrice D3FEND e produce una mappatura strutturata per ogni gap:

TTP osservata: [codice ATT&CK]
Contromisura: [codice D3FEND]
Primitiva: Harden / Detect / Isolate / Deceive / Evict
Motivazione: [testo]
Il piano rimane privato fino al Modulo 6.

Output: Piano di contromisure privato, reso pubblico solo nel reveal finale.

Modulo 4 — Strategia di inganno con MITRE Engage

Concetti da scoprire

- La logica di *adversary engagement*: guidare l'avversario verso comportamenti prevedibili;
- Categorie operative di Engage: Expose, Affect, Elicit, Lure;
- Come progettare esche plausibili in un ambiente con più attori simultanei.

Lab

Ogni team compila un **Deception Canvas**. MITRE Engage mette a disposizione strumenti propri per questa attività. La sfida è progettare esche che abbiano senso contro avversari sconosciuti — non si può tarare l'inganno su un attore specifico perché non si sa ancora chi si ha davanti.

Per ogni gap di visibilità identificato nel Modulo 3: *"Posiziono l'esca X — se qualcuno la accede, scatta l'alert Y — applico la primitiva D3FEND Z."*

Il Canvas rimane privato fino al Modulo 6.

Output: Deception Canvas privato, reso pubblico solo nel reveal finale.

Modulo 5 — Deployment: Honeypot & Deception Toolkit

Concetti da scoprire

- Criteri di plausibilità e posizionamento delle esche;
- T-Pot (honeypot multi-servizio), CanaryTokens (token traccianti);
- Segmentazione di rete su VMware (VLAN, vSwitch).

Lab

Ogni team implementa il proprio Canvas sull'ambiente VMware, su due livelli:

- **Rete:** clona T-Pot da GitHub, lo installa, assegna la VM a una VLAN dedicata tramite vSwitch;
- **File:** genera un PDF tracciante via Web GUI CanaryTokens, lo posiziona in modo credibile sul Desktop della VM target.

Ogni team documenta la logica di ogni esca e la primitiva D3FEND collegata in un file interno, senza condividerlo con gli altri. In parallelo, aggiorna il proprio profilo Caldera in preparazione allo scontro del Modulo 6.

Output: Ambiente honeypot operativo; documentazione privata delle esche.

Modulo 6 — Purple Teaming & Analisi in tempo reale

Concetti da scoprire

- Distinzione tra falso positivo, falso negativo e comportamento avversario reale;
- Triage e contestualizzazione degli alert sotto pressione con più attori simultanei;
- Attribuzione in tempo reale: separare i pattern di due attori distinti sullo stesso stack di rilevamento.

Lab — Lo scontro

Tutti e tre i team operano simultaneamente. Ogni team attacca i due ambienti avversari con il proprio profilo Caldera e difende il proprio ambiente monitorando il proprio stack di rilevamento e la dashboard T-Pot.

Sistema di punteggio

Le performance vengono valutate su più dimensioni:

Offensivo (Red Team):

- TTP eseguita senza detection → +1
- Catena di TTP coerente e completa → bonus
- Tempo prolungato senza detection (stealth) → bonus

Difensivo (Blue Team):

- Esca attivata e identificata → +1
- Tempo di risposta ridotto → bonus
- Detection con attribuzione corretta → moltiplicatore sul punteggio della detection
- Detection senza attribuzione → punteggio base, nessun moltiplicatore

Penalità:

- TTP fuori profilo → -1
- Sequenze incoerenti → -2
- Dipendenza eccessiva da capacità acquistate → penalità variabile

Economia:

- Crediti Operativi residui → bonus finale
- Crediti spesi → costo opportunità
- Crediti guadagnati → vantaggio strategico

Il punteggio finale riflette non solo l'efficacia tecnica, ma anche la gestione delle risorse e la coerenza operativa.

Come attaccante:

- Seguire esclusivamente le TTPs dell'attore assegnato;
- Cercare di non attivare le esche.

Come difensore:

- Monitorare il proprio stack di rilevamento e la dashboard T-Pot;
- Identificare le TTPs in arrivo e tentare di attribuirle a un attore;
- Reagire in tempo reale: isolare la VM target agendo sulla rete virtuale (D3FEND Isolate).

Ogni evento viene loggato in tempo reale: timestamp, esca attivata, TTP, alert, azione, tempo di risposta.

Al termine, ogni team presenta live la propria mappa ATT&CK — attore assegnato, TTPs selezionate, scelte operative. Gli altri due team confrontano in tempo reale con il proprio log: scoprono chi li stava attaccando e con quanta fedeltà sono riusciti a rilevare e attribuire le TTPs sotto pressione.

Output: Log dell'incidente; punteggi; analisi di attribuzione.

Attribuzione in tempo reale

Ogni team deve tentare di distinguere attività provenienti da attori diversi operanti simultaneamente.

L'attribuzione viene valutata su:

- riconoscimento delle TTP caratteristiche;
- coerenza della ricostruzione della kill chain;
- capacità di separare segnali sovrapposti.

Attribuzioni corrette moltiplicano il valore delle detection associate; attribuzioni errate non sono penalizzate direttamente ma abbassano il moltiplicatore e influenzano le decisioni operative.

Modulo 7 — Playbook operativi e Standard Operating Procedure

Concetti da scoprire

- Il valore delle SOP nella risposta agli incidenti;
- Struttura di un playbook visuale operativo: leggibile sotto stress, verificabile, aggiornabile;
- Ciclo di revisione post-esercitazione: lesson learned → update SOP.

Lab

Ogni team produce un **Playbook Operativo Visuale** basato sul log del Modulo 6. Struttura obbligatoria:

```
[TRIGGER — Evento rilevato / CanaryToken]
        ↓
[CHECK — Identificazione TTP su ATT&CK]
        ↓
[DECISIONE — Engage + primitiva D3FEND]
        ↓
[AZIONE — Rete virtuale / sistema target]
        ↓
[DOCUMENTAZIONE — Log, timestamp, aggiornamento SOP]
```

I playbook vengono condivisi tra tutti e tre i team per una **revisione incrociata** live, concentrandosi su scenari non coperti, decisioni ambigue e incongruenze con quanto accaduto nel Modulo 6.

Output finale del corso: Playbook operativo; Deception Canvas; mappa ATT&CK e Attack Flow dell'attore assegnato; piano D3FEND; log dello scontro.

Debrief operativo

Ogni team analizza:

- cosa pensava stesse accadendo durante lo scontro;
- cosa è accaduto realmente (alla luce del reveal);
- quali assunzioni si sono rivelate errate;
- quali decisioni sono state prese sotto incertezza.

L'obiettivo è esplicitare il modello mentale utilizzato e correggerlo.

Il valore del corso emerge principalmente da questa fase.

Riepilogo deliverable per team

Modulo	**Deliverable**	**Visibilità**
Normativo	Mappa sintetica confini legali	Pubblica

Modulo	Deliverable	Visibilità
1 — CTI & ATT&CK	Mappa ATT&CK + Attack Flow attore assegnato	Privata → pubblica al Modulo 6
2 — Caldera	Ambiente target + profilo operativo	Privato
3 — D3FEND	Piano contromisure ATT&CK → D3FEND	Privata → pubblica al Modulo 6
4 — Engage	Deception Canvas	Privata → pubblica al Modulo 6
5 — Deployment	Ambiente honeypot + documentazione esche	Privata → pubblica al Modulo 6
6 — Purple Teaming	Log dello scontro + punteggi + attribuzione	Pubblica
7 — Playbook	SOP visuale	Pubblica

Strumenti e piattaforme

Categoria	Strumento	Fonte
Framework offensivo	MITRE Caldera (agente Sandcat)	GitHub
Stack di rilevamento	Sysmon + Sigma + Chainsaw/Hayabusa — o a scelta del team	GitHub
Threat Intelligence	MITRE ATT&CK Navigator	GitHub / Web
CTI strutturata	MITRE CTI (STIX)	GitHub
Visualizzazione sequenze	Attack Flow	GitHub / Web
Contromisure	MITRE D3FEND	Web
Engagement strategy	MITRE Engage	Web
Honeypot	T-Pot	GitHub

Categoria	Strumento	Fonte
Token traccianti	CanaryTokens	Web GUI
Virtualizzazione	VMware Workstation	—

Documento soggetto a revisione. Versione corrente allineata a MITRE ATT&CK v14, D3FEND v1.0, Engage v1.0.